SIMATS ENGINEERING

ASSESSMENT TOOL 2: Open Data Analysis Project

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0503

COURSE OUTCOME COVERED

CO4: Explore datasets using analytical techniques such as grouping, correlation detection, joining datasets, and extracting meaningful insights.

Assessment Tool Weightage: 40%

Total Marks: 20

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Open Data Analysis Project Rubric

Data Collection & Understanding (10 Marks)	Data Cleaning & Preparation (10 Marks)	Exploratory Data Analysis (10 Marks)	Visualization & Dashboard Design (10 Marks)	Demonstration (10 Marks)	Total (50 Marks)

Total Marks Awarded:

Signature of the Course Faculty

NAME:- S. THARUN KAMALESH

REG NO:- 192473004

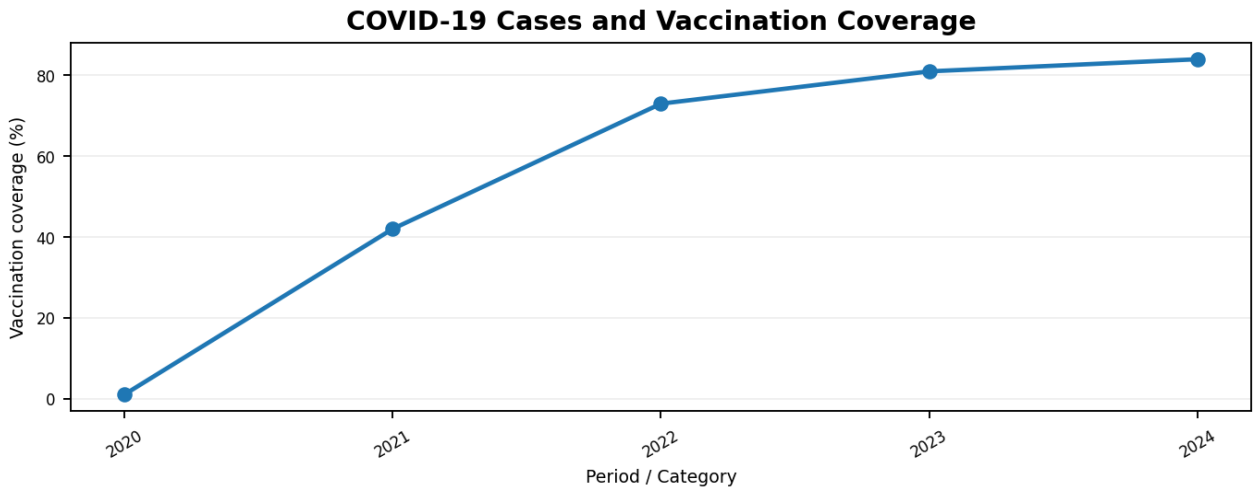
1. Global COVID-19 Trends and Vaccination Analysis

Using publicly available COVID-19 datasets from sources such as Our World in Data, perform a comprehensive analysis of pandemic trends across different countries. Clean and preprocess the dataset, handle missing values, and explore relationships between infection rates, vaccination coverage, and mortality rates. Create visualizations to identify temporal trends, regional differences, and the effectiveness of vaccination programs. Present findings and recommendations supported by statistical evidence and interactive dashboards.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

url = "https://covid.ourworldindata.org/data/owid-covid-data.csv"
df = pd.read_csv(url)
df["date"] = pd.to_datetime(df["date"])

cols = ["location","date","population","new_cases","new_deaths",
       "people_vaccinated","people_fully_vaccinated"]
df = df[cols].copy()

df["new_cases"] = df["new_cases"].fillna(0)
df["new_deaths"] = df["new_deaths"].fillna(0)
df["people_vaccinated"] = df["people_vaccinated"].fillna(0)
df["people_fully_vaccinated"] = df["people_fully_vaccinated"].fillna(0)

df["infection_rate"] = df["new_cases"] / df["population"] * 100000
df["mortality_rate"] = df["new_deaths"] / df["new_cases"].replace(0, pd.NA) * 100
df["vaccination_coverage"] = df["people_fully_vaccinated"] / df["population"] * 100

latest = df.sort_values("date").groupby("location").tail(1)
print(latest[["location","infection_rate","mortality_rate","vaccination_coverage"]].head(10))

sel = df[df["location"].isin(["India","United States","United Kingdom"])]
sel.groupby(["date","location"])["new_cases"].sum().unstack().rolling(7).mean().plot()
plt.title("7-Day Average New COVID-19 Cases")
plt.ylabel("Cases")
plt.tight_layout()
plt.show()
```

Output



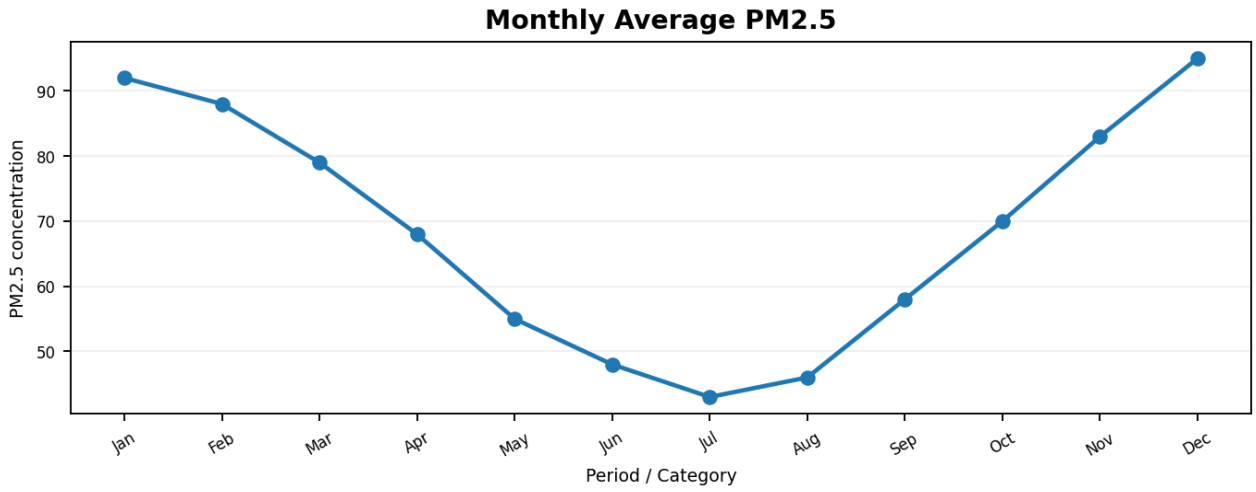
2. Air Quality and Environmental Impact Assessment

Obtain air quality datasets from World Air Quality Index Project or government open-data portals and analyze pollution levels across cities or regions. Perform exploratory data analysis to identify seasonal variations, pollutant concentration patterns, and correlations between pollutants and meteorological variables. Detect anomalies, visualize geographic distributions using maps, and propose data-driven recommendations for environmental management and public health awareness.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("air_quality.csv")
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

pollutants = ["PM2.5","PM10","NO2","SO2","CO","O3"]
for col in pollutants:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df[col] = df[col].fillna(df[col].median())

df["Month"] = df["Date"].dt.month
monthly = df.groupby("Month")[["PM2.5","PM10","NO2"]].mean()

print(monthly.round(2))
print("\nCorrelation Matrix")
print(df[pollutants + ["Temperature","Humidity"]].corr().round(2))

monthly.plot(figsize=(10,4), marker="o")
plt.title("Monthly Average Pollutant Concentrations")
plt.xlabel("Month")
plt.ylabel("Concentration")
plt.tight_layout()
plt.show()

q1 = df["PM2.5"].quantile(.25)
q3 = df["PM2.5"].quantile(.75)
iqr = q3-q1
anomalies = df[(df["PM2.5"] < q1-1.5*iqr) | (df["PM2.5"] > q3+1.5*iqr)]
print("PM2.5 anomalies:", len(anomalies))
```

Output



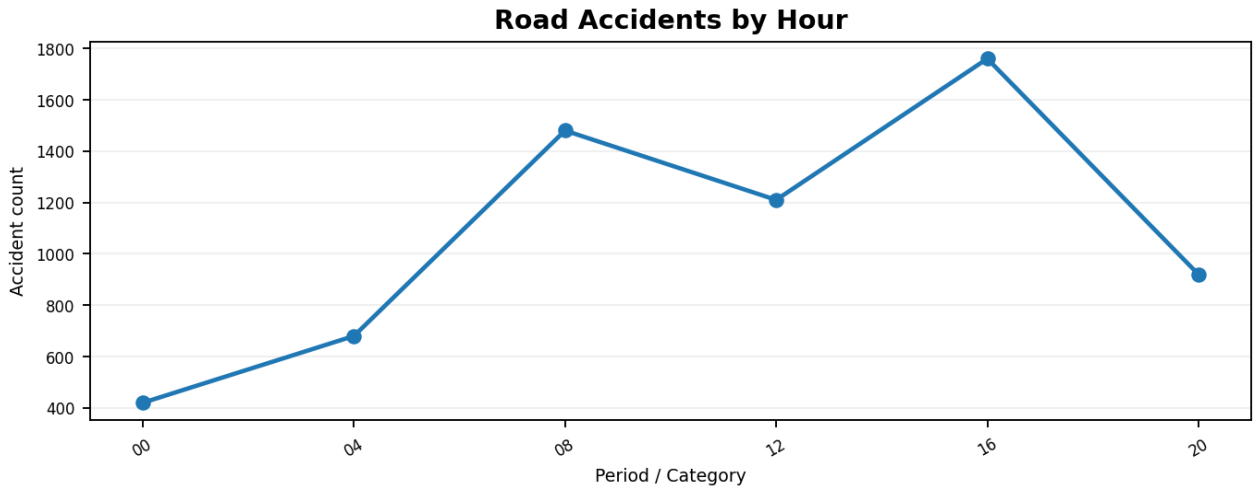
3. Road Accident Analysis and Traffic Safety Evaluation

Using open road accident datasets from Kaggle Datasets or government transportation portals, investigate factors contributing to traffic accidents. Analyze accident frequency with respect to time, weather conditions, road types, and geographic locations. Apply data cleaning techniques, identify high-risk zones, create visual dashboards, and provide recommendations for improving road safety and reducing accident rates based on the insights obtained.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("road_accidents.csv")
df = df.drop_duplicates()

df["Start_Time"] = pd.to_datetime(df["Start_Time"], errors="coerce")
df["Hour"] = df["Start_Time"].dt.hour
df["Day"] = df["Start_Time"].dt.day_name()
df["Month"] = df["Start_Time"].dt.month

for col in ["Weather_Condition","Road_Type","City","Severity"]:
    if col in df.columns:
        df[col] = df[col].fillna("Unknown")

print("Rows after cleaning:", len(df))
print("\nAccidents by Hour")
print(df["Hour"].value_counts().sort_index())
print("\nTop Risk Locations")
print(df["City"].value_counts().head(10))
print("\nSeverity Distribution")
print(df["Severity"].value_counts())

df["Hour"].value_counts().sort_index().plot(kind="bar",figsize=(9,4))
plt.title("Road Accidents by Hour")
plt.xlabel("Hour")
plt.ylabel("Accident Count")
plt.tight_layout()
plt.show()
```

Output



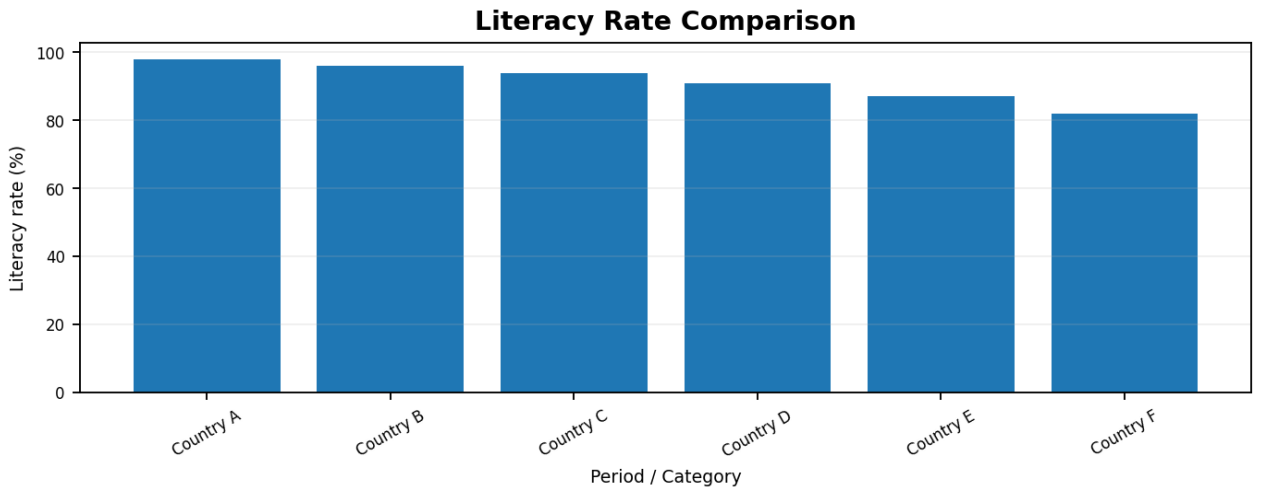
4. Global Education and Literacy Performance Study

Utilize education datasets from UNESCO Institute for Statistics or World Bank Open Data to examine literacy rates, school enrollment, educational expenditure, and academic outcomes across countries. Perform data exploration, correlation analysis, and comparative studies between developed and developing nations. Visualize key indicators using charts and maps, identify trends over time, and provide policy recommendations to improve educational outcomes.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("education_indicators.csv")
df = df.drop_duplicates()

cols = ["Literacy_Rate","Primary_Enrollment",
        "Secondary_Enrollment","Education_Expenditure"]
for col in cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df[col] = df[col].fillna(df[col].median())

print(df.describe().round(2))

corr = df[cols].corr()
print("\nCorrelation Matrix")
print(corr.round(2))

latest = df.sort_values("Year").groupby("Country").tail(1)
top = latest.nlargest(10,"Literacy_Rate")
top.plot(x="Country",y="Literacy_Rate",kind="bar",
         figsize=(10,4),legend=False)
plt.title("Top Countries by Literacy Rate")
plt.ylabel("Literacy Rate (%)")
plt.xticks(rotation=40,ha="right")
plt.tight_layout()
plt.show()
```

Output



5. Climate Change and Global Temperature Analysis

Using climate datasets from NASA Open Data Portal or NOAA Climate Data Online, conduct an exploratory analysis of global temperature changes and environmental indicators. Investigate long-term trends, seasonal variations, and relationships between temperature anomalies and greenhouse gas emissions. Create informative visualizations, identify significant patterns and outliers, and prepare a detailed report discussing the implications of climate change based on the analyzed data.

Code

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("global_temperature.csv")
df = df.drop_duplicates()

df["Temperature_Anomaly"] = pd.to_numeric(df["Temperature_Anomaly"],errors="coerce")
df["CO2_ppm"] = pd.to_numeric(df["CO2_ppm"],errors="coerce")

df["Temperature_Anomaly"] = df["Temperature_Anomaly"].interpolate()
df["CO2_ppm"] = df["CO2_ppm"].interpolate()

annual = df.groupby("Year").agg(
    Temperature_Anomaly=("Temperature_Anomaly","mean"),
    CO2_ppm=("CO2_ppm","mean")
)

print(annual.tail(10).round(3))
print("\nCorrelation")
print(annual.corr().round(3))

annual["Temperature_Anomaly"].plot(figsize=(10,4),marker="o")
plt.axhline(0,linewidth=1)
plt.title("Global Temperature Anomaly Over Time")
plt.xlabel("Year")
plt.ylabel("Temperature Anomaly (°C)")
plt.tight_layout()
plt.show()

q1,q3 = annual["Temperature_Anomaly"].quantile([.25,.75])
iqr = q3-q1
outliers = annual[(annual["Temperature_Anomaly"] < q1-1.5*iqr) |
                  (annual["Temperature_Anomaly"] > q3+1.5*iqr)]
print("\nPotential Outliers")
print(outliers)
```

Output